# Assignment 3

Kaminsky Attack

## Network Security (50.020)

**0. Objectives**

In this lab, you will learn:

- Fundamental DNS architecture and operation.
- DNS security weaknesses and attack surfaces.
- DNS spoofing and cache-poisoning concepts.
- Principles of the Kaminsky attack.
- DNS traffic analysis and attack observation.
- AI-assisted approaches for network threat detection

Lab Setup:

- VirtualBox
- 1 Kali VM (root:toor) + NAT Network
- 4 Docker containers + Internal Network

**1. Network and Basic Operations**

This lab environment consists of 4 containers connected through the same Docker network. All containers can communicate with one another and have Internet access. Each node has a specific role in the DNS experiments.

- **node1 — User: 10.11.2.5**
  Normal DNS client used to generate DNS queries and observe DNS responses.
- **node2 — Local DNS Server: 10.11.2.6**
  Recursive and caching DNS resolver used by node1 and node3.
- **node3 — Analyst: 10.11.2.7**
  Security analysis workstation used for packet analysis and experimental tasks.
- **node4 — Authoritative DNS Server: 10.11.2.8**
  Lab-controlled authoritative DNS server and web server.

Copy **lab3_docker_setup.zip** to the Kali Desktop, extract it, enter the extracted folder, and start the environment:

```
chmod +x *
./start.sh
```

The script creates the network, starts the containers, configures the services, and performs basic checks. Use the following command to view the running containers:

```
docker compose ps
```

To enter a container directly as root, press Ctrl+Alt+T to start a new terminal, then:

- docksh node1
- docksh node2
- docksh node3
- docksh node4

Inside each container, become familiar with basic commands such as:

- ls
- ifconfig
- gedit
- wireshark

**We have disabled DNSSEC by default and configured the DNS resolver to use the fixed source port 12345.**

**2. DNS and the Kaminsky Attack**

DNS translates domain names into information such as IP addresses. Normally, a client sends a query to a recursive DNS resolver. The resolver may answer from its cache or contact other DNS servers to obtain the result. DNS commonly uses UDP, and a response must correspond to an outstanding query. Important fields include the query name, transaction ID (TXID), source and destination IP addresses, and UDP ports.

In this lab, observe how the Kaminsky attack exploits repeated DNS queries and forged responses to poison a recursive resolver's cache. Follow the sequence below to understand the attack flow:

- **Force a new DNS query**: The attacker requests a random, uncached subdomain such as abc123.example.com, causing the resolver to contact an authoritative server. This behavior is discussed in RFC 5452 §4.1.
- **Race the legitimate response**: The attacker sends many forged DNS replies and attempts to guess the correct transaction ID and UDP port before the legitimate response arrives (RFC 5452 §§4.3–4.6).
- **Repeat the attack**: By using a new random subdomain for each attempt, the attacker can continuously create new query opportunities without waiting for a cached record to expire (RFC 5452 §8.1).
- **Poison the NS delegation**: The forged response attempts to insert a malicious NS record so that future queries are redirected to an attacker-controlled DNS server. RFC 3833 §2.3 explains that NS records can redirect subsequent DNS resolution, while RFC 1034 §5.3.3 specifies that delegation information learned during resolution may be cached.

Important Commands:

- dig example.com (run multiple times the first time)
- wireshark
- dumpcache | grep example.com
- flushcache

| | |
|---|---|
| | In this task, you will observe normal DNS resolution from different perspectives in the lab environment. Start Wireshark on all four nodes so that you can compare which packets are visible at each point in the network. Pay particular attention to the difference between recursive DNS resolution through the local DNS server and direct queries to an authoritative DNS server. Before generating any queries, enter node2 and inspect the local DNS cache. At the beginning of the experiment, the relevant cache entries should be empty.<br><br>On node2, inspect the DNS cache:<br>• dumpcache<br><br>On node1, run the following commands in sequence:<br>• dig example.com<br>• dig ns.attackerpage.com<br>• dig @ns.attackerpage.com example.com<br><br>Observe the corresponding traffic in Wireshark on all nodes. After completing the three queries, return to node2 and inspect the cache again:<br>• dumpcache<br><br>Then, from node1, run the first query again:<br>• dig example.com<br><br>**Question:** Based on the three dig commands, explain what you observe and describe the purpose and result of each command. Identify which DNS server is contacted and whether the query is resolved recursively or sent directly to a specified DNS server.<br><br>Inspect the DNS cache on node2 after the queries. What new DNS records can you find? Explain why these records were added to the cache and how they relate to the queries you performed.<br><br>Finally, examine the DNS transaction IDs in Wireshark. Can you identify the transaction associated with the query between node1 and node2, and the separate transaction generated by node2 for resolver-side DNS communication? Explain why the two transaction IDs are the same/different and how the correct transaction ID is restored in the response returned to node1. |
| **3.** | **Construct DNS Query and Response**<br><br>In this task, you will use AI as a programming assistant to help you understand DNS packet structure and create the required Python code. If you need additional Python packages, you may install them directly inside a running container, for example:<br>• pip3 install <package><br><br>You do not need to rebuild the Docker image. Work on node3 for the following tasks.<br><br>Create a Python program that sends a **DNS query to the local DNS server on node2**. You may use any valid query, for example, an A record query for example.com. Run Wireshark on both node2 and node3, then submit your screenshots showing that the query was successfully sent |

| | |
|---|---|
| | from node3 to node2 and that the corresponding DNS response was received. Briefly explain the important DNS and UDP fields shown in your capture.<br><br>Now, based on your Wireshark observations, explain the main challenge an attacker faces when trying to construct a forged DNS response that will be accepted by the local DNS server. Identify which information must match the outstanding DNS query. Use AI to help you create Python code that **constructs a DNS response corresponding to your query.** For example, use the following values:<br><br>Transaction ID: 0x1234 (can set randomly)<br>Query: A example.com<br>Answer: example.com A 66.66.66.66<br>Authority: example.com NS ns.attackerpage.com<br><br>Send the query first, then send the constructed response to the local DNS server and use Wireshark to compare it with the legitimate response. Compare the packet timestamps and answer: Which response reaches the local DNS server first, your constructed response or the legitimate DNS response?<br><br>If you successfully match the correct Transaction ID and your spoofed response reaches the DNS server before the legitimate response, the **Kaminsky attack will be launched successfully**. However, this is **very unlikely in this attempt**, because you send only one query and one spoofed response, while the Transaction ID search space is large. |
| **4.** | **Kaminsky Attack Demonstration**<br><br>For this task, use the instructor-provided demonstration code: **attack.py** inside the isolated lab environment. Before starting, flush node2's cache and begin a packet capture. Observe the queries and responses.<br><br>On node 3<br>• cd ~<br>• gedit attack.py<br>• python3 attack.py<br><br>**Question: Change num_responses to 20, 50, and 200,** then repeat the experiment. For each setting, observe when the first legitimate DNS query is sent and record its destination IP address. Compare how many response packets your program can send before the legitimate response arrives and determine which num_responses value gives the best experimental coverage. Based on the DNS Transaction ID search space, estimate the probability of obtaining a matching Transaction ID using this code.<br><br>**Question:** Run the experiment multiple times. You may observe that the legitimate DNS query is sent to different destination IP addresses. Identify all destination IP addresses observed during your experiments, record them clearly, and include the complete list in your report. Finally, explain at a high level how you would improve the experiment design to increase the probability of matching the correct response conditions. Focus on your reasoning and design approach; **implementation is not required.** |

| | |
|---|---|
| **5.** | **AI-Assisted Detection of DNS Attacks**<br><br>Use AI to develop a Python-based real-time Kaminsky attack detector on Node2. Run the attack program from Node3 and take a screenshot showing that the detector works well. Briefly explain the traffic features monitored by the detector, why these features may indicate a Kaminsky attack, and why the chosen detection design is effective. |
| **6.** | **Submission**<br><br>• 1 PDF Report: Include screenshots, answers, analysis results, and brief reflection on your AI experience and learning<br>• Code Files: Scripts used in the lab. |